

HPC2526 - Consegna 1: PCsetup

Obiettivo: configurare il dispositivo personale (PC o laptop) per le attività di laboratorio

La prima parte delle esercitazioni si svolgeranno su Personal Computer (PC) se disponibile. Chi non dispone di un PC con le risorse adeguate attenderà l'attivazione degli account sul cluster di Ateneo.

Configurazione “Client”

Per l'utilizzo del cluster di Ateneo sarà necessaria una configurazione “client” del PC, con un client SSH e un client VPN. La configurazione del client si farà quando sarà abilitato l'accesso al cluster.

Configurazione “Nodo di calcolo – Worker Node (wn)”

Creare sul PC la cartella di lavoro **HPC2526/** per le attività del corso.

Per eseguire i programmi sul PC personale esistono diverse alternative, in base al sistema operativo del PC.

Windows10/Windows11: utilizziamo container Docker con Ubuntu e dotati dei pacchetti necessari per la compilazione, l'esecuzione e l'analisi dei risultati. E' quindi necessaria l'installazione di [Docker](#) sul PC.

Su DockerHub è disponibile l'immagine **retilab/ubuntu-hpc** derivata dall'immagine Ubuntu a cui è stato aggiunto l'ambiente software necessario. Il **dockerfile** utilizzato per la generazione dell'immagine è disponibile in questa consegna, a scopo di consultazione.

Nella consegna si trova anche il **compose.yml** che attiva due container wn1 e wn2, con l'utente root la cui home dir /root/ è condivisa con la directory HPC2526 dell'host (esempio C:\HPC2526).

Il file compose.yml aggiunge anche i driver CUDA per la GPU NVIDIA. Se il PC non ha una GPU NVIDIA occorre rimuovere il caricamento dei driver.

Oltre all'installazione di [Docker Desktop per Windows](#) si consiglia l'installazione di **MobaXterm** <https://mobaxterm.mobatek.net/>

L'interfaccia utente per Docker può essere quella grafica di Docker Desktop o a linea di comando utilizzando l'interfaccia "Shell". Noi utilizzeremo la Shell di MobaXterm. *[Video \(relativo al corso Reti\) per la configurazione Shell di MobaXterm](#)*

Linux o Mac: Se il sistema operativo è Unix-Like, come Linux o MacOS, oltre all'installazione e configurazione di Docker è possibile, in alternativa, installare nativamente l'ambiente software. Per Linux si può fare riferimento al dockerfile, mentre per MacOS nella consegna è disponibile la guida **guida_porting_gcc_clang**.

Configurazione multinodo

Se l'installazione è via Docker potremo configurare due Worker Node, wn1 e wn2, utili per i test di programmazione a memoria distribuita (con MPI).

Attivazione dei due container e accesso interattivo a wn1:

```
docker compose up -d
docker compose exec wn1 bash
```

Accesso password-less tra i nodi; per il funzionamento di MPI è necessario che l'utente possa accedere via ssh tra i **wn** senza password, utilizzando autenticazione RSA.

La directory /root deve appartenere a root ed essere protetta. In caso:

```
chown root:root /root
chmod 755 /root
```

Solo la prima volta:

```
mkdir /root/.ssh # se necessario
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

Verifica da wn1: `ssh wn2` non richiede password

Test del Worker Node del PC

Per questo primo laboratorio creare la directory HPC2526/1-PCsetup/ in cui copiate i file allegati in questa consegna.

Il tar DeviceQuery.tgz contiene programmi per raccogliere informazioni sulle caratteristiche hardware di CPU e GPU, sia sul PC che cluster di Ateneo.

Sul cluster il tar file può essere copiato con il seguente comando:

```
cp /hpc/home/roberto.alfieri/SHARE/HPC2526/deviceQuery.tgz .
```

Per scompattare: `tar xzvf deviceQuery.tgz`

All'interno del tar gli script con estensione .slurm serviranno per i run sul cluster di Ateneo (`sbatch <script>.slurm`), ma generalmente potranno essere eseguiti anche sul PC (`sh <script>.slurm`)

CPU: sh lsCPU.sh (oppure sh lsCPU.slurm)

GPU: sh deviceQuery.sh (oppure sh deviceQuery.slurm)

Consegna:

1) Descrivere l'installazione eseguita, incluso in sistema operativo e il tipo di GPU.

2) Con i comandi di test raccogliere informazioni sulla configurazione del nodo di calcolo:

CPU: numero di core, frequenza di lavoro, dati su memoria e cache, presenza di istruzioni SIMD (AVX, AVX2, AVX512, ..) e FMA.

GPU: Cuda driver version, dimensione memoria globale, memoria shared, numero di cuda-core, max clock rate.

3) Determinare le prestazioni di picco in Flops:

1 core CPU: frequenza massima x 2 (FMA)* x N (SIMD)**

1 nodo CPU: prestazioni di 1 core x numero di core fisici (no hyper-threading)

1 core GPU: frequenza massima x 2 (FMA)

1 GPU: prestazioni di 1 core x numero di core

**) FMA (Fused Multiply Add) è una istruzione che in una CPU o GPU consente di eseguire contemporaneamente una somma e una moltiplicazione floating point*

https://en.wikipedia.org/wiki/Multiply%E2%80%93accumulate_operation#Fused_multiply

***) N=16 se la CPU supporta AVX512, N=8 per AVX2, ecc*

https://en.wikipedia.org/wiki/Floating_point_operations_per_second

Scrivere la relazione in relazione1.pdf (o relazione1.txt) da caricare su Elly.